# Control Strategy: Nov27th Path Correction Code

Oct 28, 2026 · @Rasitha R

## Overview

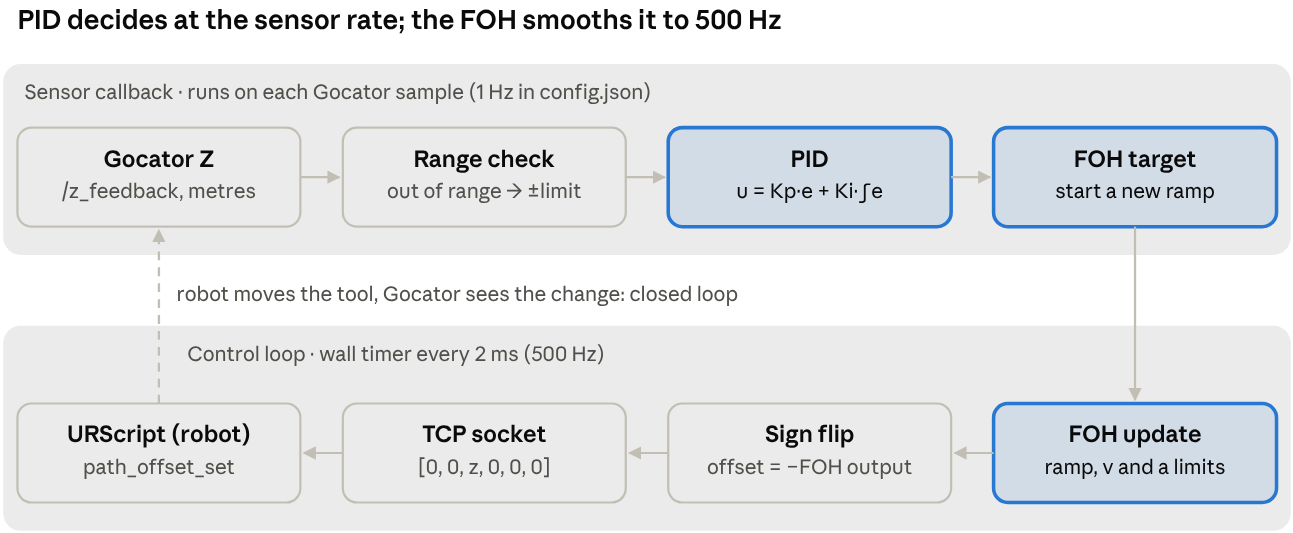
The Nov27th code is a single-axis (Z height) closed-loop correction: a PID turns the Gocator height error into an offset at the sensor rate, and a rate- and acceleration-limited first-order hold (FOH) stretches that offset into a smooth 500 Hz stream sent to the UR10e. It runs, but with the shipped gains the loop is unstable and several safety and timing issues need fixing before production use.

What runs today (`src/pathcorrection.cpp`, built as `dynamicontrol`):

- **Input:** Gocator Z value on ROS 2 topic `/z_feedback` (`std_msgs/Float32`, metres).
- **Control law:** PID on `error = setpoint − sensor_z`, with output clamping and conditional anti-windup.
- **Signal shaping:** FOH ramps from the previous command to the new PID target over one sensor period, with velocity and acceleration limits.
- **Output:** every 2 ms, `-FOH output` is sent over a TCP socket as `[0, 0, z, 0, 0, 0]`; a URScript program on the robot (not in the zip) applies it with `path_offset_set`.
- **Safety:** if the reading leaves the Gocator range (±12.5 mm), the command jumps to the offset limit (±30 mm).
- **Logging:** sensor, PID and FOH values every 2 ms, dumped to JSON or CSV on shutdown.

Older prototypes (`pathcorrection_thread.cpp`, `scripts/path_correction.py`) are in the zip but are not built and do not match the current headers.

# Architecture and data flow




Nov27th data flow · two rates, one loop

The sensor callback and the 2 ms timer run in one single-threaded ROS 2 executor and share only the FOH target. The loop closes physically: the robot moves the tool, so the next Gocator reading already contains the applied offset.

# Design strategy: why a PID and why an FOH

The design splits the problem into two rates: a slow "decide" loop that runs whenever a Gocator sample arrives, and a fast "actuate" loop that feeds the robot every 2 ms. The PID decides; the FOH bridges the gap between the two rates.

## Why a PID

- **The sensor measures the result of the correction.** The Gocator rides on the flange, so after the robot moves, the next reading already includes that move. That is a closed loop, and a PID is the standard tool for driving a measured value (sensor Z) to a setpoint (−8 mm).
- **Integral action removes steady error.** If the part sits 2 mm lower over a long stretch, the integral keeps adding offset until the reading equals the setpoint again.
- **Proportional action reacts quickly** to a new error; derivative action was planned (filtered D with coefficient N) but is switched off (Kd = 0).

- **Output limits** keep the commanded offset inside ±30 mm (config), protecting the part and the robot.

### Why an FOH

- **Rate mismatch.** The Gocator data arrives at 1–30 Hz (config: 1 Hz or 5 Hz); `path_offset_set` is applied at 500 Hz. Sending a new PID value only when it changes would produce a staircase: a step every sensor period.
- **Steps cause protective stops.** UR rejects offsets that change too fast (C219 path-offset faults), so the command must be smooth.
- **First-order hold = straight-line interpolation.** Instead of jumping to the new PID value, the FOH ramps from the previous value to the new one over one sensor period, so the robot sees a continuous, piecewise-linear command.
- **Extra protection.** On top of the ramp, the FOH caps the offset velocity ( `max_rate_limit` ) and acceleration ( `max_accel_limit` ), so even a large PID step or a fail-safe command is applied gently.

### Why not a zero-order hold (ZOH)

A `ZeroOrderHold` class exists and CMake has a `USE_ZOH` option, but the ZOH source file ( `pathcorrection_zoh.cpp` ) is not in the zip. A ZOH holds each value until the next sample, giving steps at every sample, which is exactly what the FOH avoids.

### The price of this design

The FOH reaches each new target only at the end of the sensor period, so it adds about one sensor period of delay (1 s at the shipped 1 Hz). In a closed loop, extra delay plus high gain leads to oscillation; see Shortcomings.

## PID design in detail

`include/utils/PIDController.hpp` implements a positional PID with automatic time step, a filtered derivative and conditional anti-windup. It runs once per Gocator sample, inside `sensorCallback` .

## Error and terms

```
error  = setpoint − sensor_z            (metres; setpoint = −0.008 in
config.json)
P      = Kp · error                     (Kp = 5)
I      = Ki · Σ(error · dt)             (Ki = 2)
D      = Kd · filtered d(error)/dt       (Kd = 0, so off)
output = clamp(P + I + D, minOffset, maxOffset)   (±0.03 m)
```

The output is an **absolute offset** (not an increment): each sample replaces the previous target.

## Time step

- `useChrono = true`: dt is measured with `rclcpp::Clock()` between consecutive calls, so it equals the real gap between Gocator messages.
- dt ⩽ 0 is replaced by 1e-6 s to avoid division by zero.
- The clock starts in the constructor, so the first dt includes the start-up time (see Shortcomings).

## Derivative filter

First-order low-pass on the raw derivative:

```
alpha              = dt / (dt + 1/N)          (N = derivative_filter_N = 10)
derivative_filtered = alpha · raw + (1 − alpha) · previous
```

With N = 10 the filter time constant is 0.1 s. It has no effect today because Kd = 0.

## Saturation and anti-windup

- Output is clamped to [minOffset, maxOffset].
- If the output saturates **and** the error pushes further into saturation, the integral update of this step is undone (`integral −= error · dt`). This is conditional integration: the integral stops growing while the output is pinned.

## Loop gain

Because the Gocator moves with the robot, a 1 mm offset change changes the reading by about 1 mm. Per sample, the PID therefore applies roughly **Kp + Ki·T** times the error:

| Config | Kp | Ki | Sensor period T | Effective gain per sample | Expected behaviour |
|---|---|---|---|---|---|
| config.json | 5 | 2 | 1.0 s | ≈ 7 | Strong oscillation (simulated ±5 mm on a 2 mm step) |
| config_Sensor-5Hz.json | 1 | 1 | 0.2 s | ≈ 1.2 | Mild oscillation (simulated ±0.7 mm) |
| Stable range | 0 | any | T | 0.3–0.7 | Converges without overshoot |

The simulations are from an earlier review of this code (static 1:1 plant plus the FOH, no robot dynamics); they show the trend, not exact amplitudes.

## Call sequence and timing

1. A Gocator message arrives on `/z_feedback`; `sensorCallback` runs.
2. If the reading is inside ±12.5 mm, `pid_->update(setpoint_z_, sensor_z)` is called once; otherwise the PID is skipped (fail-safe).
3. Inside `update()`: measure dt since the last call → compute the error → P, I, filtered D → sum → clamp → anti-windup correction → store `prevError` and `last_output_`.
4. The result is stored in `latest_pid_output_` (for logging) and handed to the FOH with `setTargetValue()`.
5. The PID is idle until the next Gocator message; the 500 Hz loop never calls it.

So the PID runs at the Gocator rate, its time step is the real gap between messages, and a missing message simply means no update (the integral does not grow during gaps).

## Worked example (config.json values)

Setpoint = −8.0 mm, Gocator reads −6.0 mm, one second since the last sample, integral starts at 0:

| Step | Calculation | Result |
|---|---|---|
| Error | −0.008 − (−0.006) | −0.002 m (−2 mm) |
| P term | 5 × (−0.002) | −0.010 m |
| Integral | 0 + (−0.002 × 1.0) | −0.002 m·s |
| I term | 2 × (−0.002) | −0.004 m |

| Step | Calculation | Result |
|---|---|---|
| D term | Kd = 0 | 0 |
| Output | −0.010 + (−0.004), inside ±0.03 | −0.014 m (−14 mm) |
| Sent to robot | × −1 after the FOH | +14 mm |

A 2 mm height error produces a 14 mm move. Because the Gocator moves with the robot, the next reading swings about 12 mm the other way, which is why the loop oscillates (Shortcomings #1).

### Strengths of the current PID design

- **Clear, standard structure:** P, I and D are separate, easy to read and to tune; gains come from config, not code.
- **Real time step:** dt is measured, so the integral stays correct even if the Gocator rate drifts or a sample is late.
- **Output saturation:** the command can never exceed ±maxOffset, which bounds the worst case.
- **Anti-windup built in:** conditional integration stops the integral from growing while saturated, so recovery after a long saturation is quick.
- **Derivative already filtered:** if D is ever switched on, sensor noise is not amplified unfiltered (N = 10).
- **Safe division:** dt ⩽ 0 is guarded.
- **Inspectable:** `getLastOutput()` and the logging make every PID output visible for tuning.
- **Reusable:** header-only class with no ROS topics inside, so it can be unit-tested on its own.

## FOH design in detail

`include/utils/FirstOrderHold.hpp` is not a pure first-order hold: it is a linear ramp with a velocity limit and an acceleration limit, effectively a small trajectory generator. `setTargetValue()` is called by the sensor callback; `update()` is called by the 500 Hz control loop.

## When a new target arrives (`setTargetValue`)

```
previous_value = current_value     (start the new ramp from where the output
is now)
target_value   = new PID output
previous_time  = now
```

## Every 2 ms (`update`)

1. **Desired velocity**
   - Within one sensor period of the new target: `v_des = (target − previous) / T_sensor` (the straight-line FOH slope).
   - After the sensor period has passed (late or missing sample): `v_des = (target − current) / T_act`, i.e. try to close the remaining gap in one 2 ms step.
2. **Velocity limit:** `v_des = clamp(v_des, ±max_rate_limit)` (0.01 m/s = 10 mm/s in config).
3. **Acceleration limit:** the velocity may change by at most `max_accel_limit · T_act` per step (1.2 m/s² × 0.002 s = 2.4 mm/s per step).
4. **Integrate:** `current += v · T_act`; return `current`.

## What this means in practice

| Situation | Behaviour |
|---|---|
| Small change (e.g. 1 mm at 1 Hz) | Straight ramp over 1 s at 1 mm/s; target reached at the end of the period |
| Large change (e.g. 30 mm fail-safe) | Capped at 10 mm/s, so it takes about 3 s; the ramp outlasts the sensor period and continues in "catch-up" mode |
| Ramp start/end | Velocity ramps up and down at 1.2 m/s² (about 8 ms to reach 10 mm/s), so no velocity steps |
| Missing samples | After one period, it keeps moving toward the last target and then holds it |
| Near the target in catch-up mode | Desired velocity is error/0.002 s, limited by the acceleration cap; can overshoot by a few hundredths of a mm and settle |

## Units

All values are metres and m/s. Comments in the file say "mm" in places (`new_target is in mm`); the code itself works in metres, consistent with config.json.

## Sign

`controlLoop()` multiplies the FOH output by −1 before sending. The sign convention between Gocator Z, the PID error and the robot offset frame is therefore fixed in code; it must match the robot script's frame (base or TCP), which is not in the zip.

## Call sequence and timing

1. **Sensor callback** (Gocator rate): `foh_->setTargetValue(pid_output)` stores the new target, freezes the start point (`previous_value = current_value`) and restarts the ramp clock.
2. **Control loop** (every 2 ms): `foh_->update()` computes the desired velocity, applies the velocity and acceleration limits, integrates one step and returns the new offset.
3. The loop multiplies the result by −1, logs it and sends it to the robot.

The FOH holds its own state (`current_value_`, `current_velocity_`), so the robot command is continuous in position and velocity even when the PID target jumps.

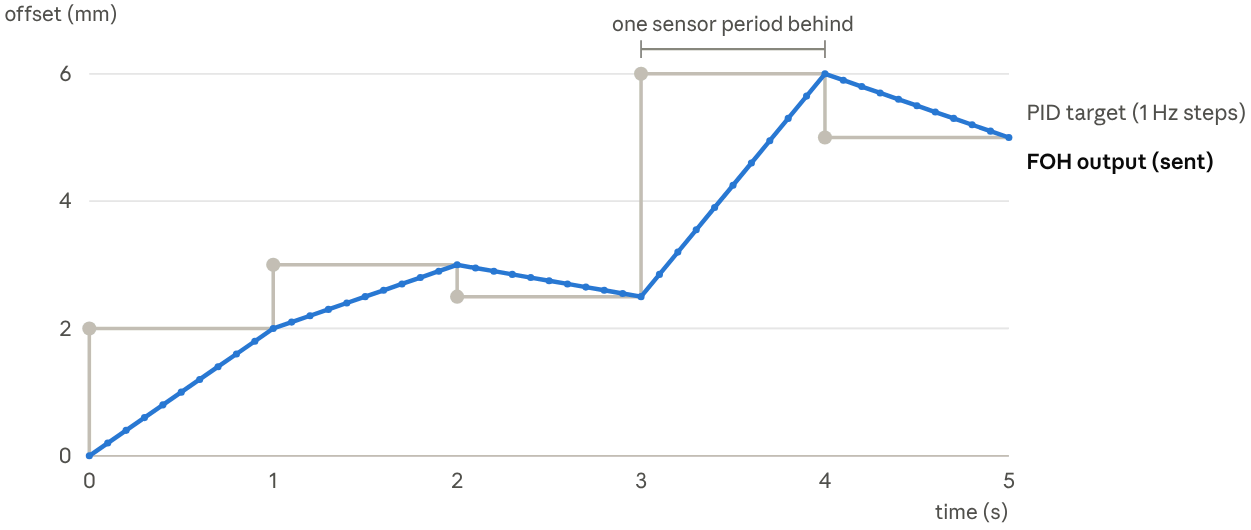
## Worked examples (config.json: 1 Hz, 10 mm/s, 1.2 m/s²)

| Case | What the FOH does | Time to reach target |
|---|---|---|
| PID target 0 → 2 mm | Ramp at 2 mm/s (below the 10 mm/s cap); velocity reaches 2 mm/s within one 2 ms step | 1.0 s (end of the sensor period) |
| PID target 2.5 → 6 mm | Ramp at 3.5 mm/s | 1.0 s |
| Fail-safe 0 → 30 mm | Wants 30 mm/s, capped at 10 mm/s; continues in catch-up mode after the first second | about 3.0 s |
| No new sample | Finishes the current ramp, then holds | — |

The chart below shows the FOH output for a sequence of 1 Hz PID targets (simulated with the FOH code's logic and the config.json limits).

**The FOH reaches each PID target one sensor period later, as a straight ramp**

Offset in mm vs time in s · config.json: 1 Hz sensor, 10 mm/s and 1.2 m/s² limits




Simulated with the FOH logic from FirstOrderHold.hpp and config.json limits · 5 PID targets at 1 Hz

The ramp keeps the robot command smooth, but each PID decision only fully reaches the robot one second later, and inside a closed loop that delay feeds the oscillation described in Shortcomings #1.

## Strengths of the current FOH design

- **Removes steps:** a 1–30 Hz decision becomes a continuous 500 Hz command, avoiding the staircase a zero-order hold would send.
- **Continuous velocity:** the acceleration limit means even the ramp corners are rounded, which is what the UR path-offset checks (C219) want.
- **Hard speed cap:** no matter what the PID or the fail-safe asks for, the offset cannot move faster than `max_rate_limit`. This is the main reason a bad target is not dangerous instantly.
- **Handles late samples:** the catch-up mode keeps moving toward the last target and then holds, instead of extrapolating into nowhere.
- **Starts from where it is:** each new ramp starts at the current output, so overlapping or early targets never cause a jump.
- **Configurable:** sensor period, velocity and acceleration limits come from config.
- **Reusable as a final shaping stage:** in the target architecture (Kalman filter + look-ahead + geometric offset), this same limiter can stay as the last step before the robot.

# Other conditions and criteria handled

| Condition | How the code handles it | Where |
|---|---|---|
| Sensor reading out of range (below −12.5 mm or above +12.5 mm) | Skips the PID and commands the full offset limit: below range → `minOffset`, above range → `maxOffset` (then ×−1 on output). The PID state is left untouched | `sensorCallback` |
| Offset limit | PID output clamped to ± `maxOffset` (±30 mm) | `PIDController::update` |
| Integral windup | Conditional integration while saturated | `PIDController::update` |
| Smooth robot motion | FOH ramp + velocity and acceleration limits | `FirstOrderHold::update` |
| Late or missing Gocator samples | FOH finishes the ramp, then holds the last target; the PID is not called | `FirstOrderHold::update` |
| Rate mismatch (sensor vs robot) | Separate callback (sensor rate) and 2 ms wall timer (500 Hz) | constructor |
| Thread safety | Shared values in `std::atomic`; log vector behind a mutex. With the default single-threaded executor, callbacks never run at the same time, so this is a safeguard | class members |
| Configuration | All gains, limits and rates read from `config/config.json` at start-up, with built-in defaults | constructor |
| Data logging | Sensor Z, PID output and FOH output stored every 2 ms; written to JSON or CSV in the destructor | `controlLoop`, `dump_data` |

| Condition | How the code handles it | Where |
|---|---|---|
| Robot connection | The PC opens a TCP server (port 50003) and blocks until the robot connects | `initServer`, `acceptConnection` |

Not handled: look-ahead distance between Gocator and nozzle, timestamps, robot pose feedback, wrong-feature or cutout readings, more than one axis, reconnection, and a watchdog on the PC side.

## Configuration parameters

The node reads a hard-coded path (`/home/ws/Nov_27th/.../config/config.json`). Defaults apply when a key is missing.

| Key | config.json | config_Sensor-5Hz.json | Code default | Effect |
|---|---|---|---|---|
| setpoint | −0.008 m | 0.0 m | 0.05 m | Gocator Z the loop drives to |
| Kp | 5.0 | 1.0 | 1.0 | Proportional gain |
| Ki | 2.0 | 1.0 | 0.5 | Integral gain (1/s) |
| Kd | 0.0 | 0.0 | 0.0 | Derivative gain (off) |
| derivative_filter_N | 10 | 10 | 10 | D-filter bandwidth (1/s) |
| minOffset / maxOffset | ±0.03 m | ±0.02 m | ±0.02 m | PID clamp and fail-safe command |
| actuating_rate | 500 Hz | 500 Hz | 500 Hz | Control-loop and send rate |
| sensor_rate | 1 Hz | 5 Hz | 30 Hz | FOH ramp period (must match the real Gocator rate) |
| max_rate_limit | 0.01 m/s | 0.01 m/s | 1.5 m/s | FOH velocity cap |
| max_accel_limit | 1.2 m/s² | 1.2 m/s² | 25 m/s² | FOH acceleration cap |

| Key | config.json | config_Sensor-5Hz.json | Code default | Effect |
|---|---|---|---|---|
| sensor_range_min / max | ±0.0125 m | ±0.0125 m | ±12.5 (no unit) | Out-of-range fail-safe threshold |
| ROBOT_IP / ROBOT_PORT | 169.254.0.1 / 50003 | same | same | Address the PC server binds to (the robot connects to it) |
| logging_mode | json | csv | csv | Dump format |

Two defaults are in the wrong unit: the sensor range default (12.5) is millimetres while everything else is metres, and the default rate and acceleration limits (1.5 m/s, 25 m/s²) are far above what the config uses.

## Shortcomings

The most serious issue is loop stability: with the shipped gains the correction oscillates by several millimetres. Next come safety (the fail-safe and start-up jumps) and timing.

| # | Shortcoming | Impact | Where |
|---|---|---|---|
| 1 | **Loop gain far above 1.** Kp = 5, Ki = 2 at 1 Hz gives about 7× the error per sample on a plant whose gain is 1. The FOH's one-period delay makes it worse | Sustained oscillation (simulated ±5 mm); 5 Hz config still about ±0.7 mm | config.json, PID |

| # | Shortcoming | Impact | Where |
|---|---|---|---|
| 2 | **Fail-safe commands the full offset.** An out-of-range or invalid Gocator value (including "no edge found") sends the robot toward ±30 mm | Nozzle can be driven into or away from the part on a single bad reading; direction depends on the swapped min/max and the ×−1 | `sensorCallback` |
| 3 | **Start-up jump.** The PID clock starts in the constructor, before `acceptConnection()` blocks; the first dt includes that wait, so the first output saturates | First command after start ramps toward the offset limit | PID constructor |
| 4 | **No look-ahead.** The reading taken L ahead of the nozzle is applied at the nozzle immediately | Height error ≈ slope × L on slopes and bumps; wrong position on curves | design |
| 5 | **1 Hz sensor rate.** The loop decides once per second and the FOH adds another second of delay | Slow response; large following error on changing surfaces | config.json |
| 6 | **Output quantised to 0.1 mm.** `setprecision(4)` in metres | Staircase command; small steps at 500 Hz | `sendOffset` |

| # | Shortcoming | Impact | Where |
|---|---|---|---|
| 7 | **Message format and robot script unknown.** Sends `[0, 0, z, 0, 0, 0]` (square brackets); `socket_read_ascii_float` expects parentheses. The URScript side is not in the zip | Parsing may fail or behave differently than intended; frame (base/TCP) unverified | `sendOffset` |
| 8 | **TCP backlog risk.** PC wall timer and robot read loop are not synchronised; two `std::cout` lines per 2 ms cycle add jitter | Stale offsets queue up in the socket; latency grows | `controlLoop`, `sendOffset` |
| 9 | **No timestamps or robot feedback.** Readings are not matched to the robot pose; the applied offset is not read back | Cannot compensate latency or verify what the robot did | design |
| 10 | **No validation of readings.** Any in-range value is trusted, including cutouts, holes and nearby edges | Wrong-feature lock; bead follows the wrong edge | design |
| 11 | **No PC watchdog or reconnection.** Socket errors are printed and ignored; `initServer` / `acceptConnection` results are not checked | Silent failure; robot keeps the last offset | constructor, `sendOffset` |
| 12 | **Unit inconsistencies.** Defaults in mm vs m; comments say mm | Wrong limits if a key is missing; confusion during tuning | constructor, FOH comments |

| # | Shortcoming | Impact | Where |
|---|---|---|---|
| 13 | **Unbounded logging.** 500 samples/s kept in memory and only written in the destructor | Lost on a crash; memory grows on long runs | `controlLoop` |
| 14 | **Hard-coded paths.** Config path and dump paths are fixed | Breaks when the workspace moves | constructor, dump functions |
| 15 | **Dead and mismatched code.** `pathcorrection_thread.cpp` and the Python node don't match current headers; the dummy publisher uses another topic, type and unit (mm); `USE_ZOH` points to a missing file; unused globals and includes | Confusing; the dummy cannot test the real node | repo |
| 16 | **Python prototype sends `path_offset_set` to port 30002.** Script sent there replaces the running robot program | That approach cannot run alongside the nominal path | `scripts/utils/ur_socket.py` |

## Improvements required

Fix stability and safety first; they need only small code changes. Then move to the architecture that removes the root causes: a look-ahead edge map with a filtered estimate, computed in base coordinates.

### Step 1: quick fixes (days)

1. **Make the loop stable.** Use integral-only control with a per-sample gain below 1: `Kp = 0`, `Ki ≈ 0.5 × sensor_rate` (0.5 at 1 Hz, 2.5 at 5 Hz). Equivalent: `offset += k · error` with k = 0.3–0.7.
2. **Replace the fail-safe.** On an out-of-range or invalid reading: hold the last offset, count it, and stop dispensing after N consecutive invalid readings. Never command the full limit.

3. **Fix the start-up jump.** Reset the PID clock on the first sample, or use a fixed `dt = 1 / sensor_rate`.
4. **Raise the sensor rate** to what the Gocator actually delivers (20–100 Hz) and set `sensor_rate` to match.
5. **Send 6 decimals** and remove the per-cycle `std::cout`.
6. **Fix units and defaults** (all metres), check return values of `initServer` and `acceptConnection`, and make paths parameters.
7. **Verify the robot side:** message format vs `socket_read_ascii_float`, frame type (1 or 2), `path_offset_set_max_offset`, alpha filter and a watchdog.

### Step 2: robust interface (weeks)

1. **Use RTDE registers** instead of the raw socket: the robot always reads the latest value (no backlog), and the actual TCP pose and applied offset can be read back.
2. **Timestamp every profile** and match it to the robot pose at capture time.
3. **Add a sign-and-frame test** (+2 mm on one axis at a time) to commissioning.

### Step 3: target architecture

1. **Look-ahead buffer:** convert each reading to base coordinates and store it by path position; use the value at the nozzle position, not the newest one.
2. **Kalman filter on the deviation from nominal** with a gate capped below the nearest wrong-feature distance, re-acquire after N rejections, coasting across cutouts.
3. **Geometric offset law:** `offset = inverse(nominal) ∘ edge frame ∘ setpoint`, then clamp, rate- and acceleration-limit (the current FOH logic can be reused here as the shaping stage).
4. **Extend axes** one at a time: Z, then X (lateral), then Rz, then Rx/Ry.
5. **Operating states:** pre-scan, tracking, coasting, re-acquire, fault stop, done.

These are described in the companion document "Control Design: Dynamic Path Offset for Sealant Dispensing".

### What to keep

- The two-rate structure (decide at sensor rate, actuate at 500 Hz).
- The FOH's velocity and acceleration limiting, as the final shaping stage.
- Config-driven parameters and the logging, written to disk continuously instead of only at shutdown.

# Files reviewed

| File | Role | Built? |
|---|---|---|
| `src/pathcorrection.cpp` | Main ROS 2 node: sensor callback, PID, FOH, 500 Hz loop, logging | Yes (`dynamicontrol`) |
| `include/utils/PIDController.hpp` | PID with auto dt, filtered D, anti-windup | Yes |
| `include/utils/FirstOrderHold.hpp` | Ramp with velocity and acceleration limits | Yes |
| `include/utils/ZeroOrderHold.hpp` | Sample-and-hold (included, unused) | Header only |
| `src/utils/socket_utils.cpp`, `include/utils/socket_utils.hpp` | TCP server/client, offset message formatting | Yes |
| `config/config.json`, `config/config_Sensor-5Hz.json` | Gains, limits, rates | Read at start-up |
| `CMakeLists.txt`, `src/CMakeLists.txt` | Build; `USE_ZOH` option points to a missing file | — |
| `src/pathcorrection_thread.cpp` | Older 30 Hz prototype, client to port 30002 | No (does not match current headers) |
| `scripts/path_correction.py` + `scripts/utils/*.py` | Python prototype with PID, FOH, ZOH, socket | No |
| `scripts/gocator_publisher_dummy.py` | Test publisher (0–10 "mm" pulse on `/sensor/z_feedback`, Float64) | Does not match the node's topic, type or unit |
| `scripts/plot_control_data.py` | Plots the CSV dump | — |

Not in the zip: the URScript program running on the robot, the Gocator/Vision node that publishes `/z_feedback`, and any test data.